



awk

NCSC101

Prof. Pranay Kumar Saha

September 1, 2025





What is AWK?

A Data Extraction & Reporting Tool

AWK is a powerful command-line utility that processes text files line by line. It excels at handling structured, column-based data.

Core Syntax:

```
1 awk 'pattern { action }' filename
```

```
2
```

- **Pattern:** A condition to select which lines to process.
- **Action:** The command(s) to execute on matched lines.



What is awk?

- Pattern-action language for scanning and processing text.
- Operates record-by-record (default record = line). Fields split by FS (default whitespace).
- Great for logs, CSV/TSV, quick reports, and data wrangling.

Code

```
1 echo "hello world" | awk '{ print $0 }'
```

Output

```
hello world
```



Lab Question

#id	user	item	amount	status
101	u01	desk-lamp	39.99	paid
102	u91	chair	129.00	refund-requested
103	u77	standing-desk	499.00	pending
104	u01	hdmi-cable	9.50	paid

```
sed -E '1b; s/^(([^ \t]*\t){2})([^\t]+)/\1\U\3/' orders.tsv > orders_upper.tsv
```

Explanation. The regex `^(([^ \t]*\t){2})([^\t]+)` captures: (1) the first two tab-separated fields, (3) the 3rd field (= *item*). Replacement `\1\U\3` keeps the prefix and uppercases only field 3. `1b` skips the header row (line 1). Output is written to `orders_upper.tsv`.



The Core Model: pattern { action }

File Content (data.txt)

```
alice 10  
bob 20  
carol 15
```

Code

```
1 awk '$2 > 10 { print $1, "passed" }' data.txt
```

Output

```
bob passed  
carol passed
```

Explanation

This is the fundamental structure of **awk**. For each line in a file, it checks if the **pattern** is true. If it is, **awk** executes the corresponding **action**. In this example, the pattern is `$2 > 10`.



Printing Columns

File Content (data.txt)

```
alice 10  
bob 20
```

Code

```
1 awk '{ print $1 }' data.txt
```

Output

```
alice  
bob
```

Explanation

`awk` automatically splits each line into fields. You can access them using `$1`, `$2`, etc. `$0` represents the entire line.



Field Separator (-F and FS)

File Content (data.csv)

```
name,score  
alice,10  
bob,20
```

Code

```
1 awk -F, 'NR>1 { print $1 " -> " $2 }' data.csv
```

Output

```
alice -> 10  
bob -> 20
```

Explanation

Use the **-F** option to specify a field separator, like a comma for CSV files. The pattern **NR>1** is used here to skip the header row.



Built-in Variables

Code

```
1 awk '{ print "NR=" NR, "NF=" NF, "First=" $1, "Last=" $NF }' data.txt
```

File Content (data.txt)

```
a b c  
d e f g
```

Output

```
NR=1 NF=3 First=a Last=c  
NR=2 NF=4 First=d Last=g
```

Explanation

awk has many built-in variables: **NR** (total record number), **NF** (number of fields in the current line), and **\$NF** (the value of the last field).



Record/Output Separators

File Content (data.txt)

```
alice 10  
bob 20
```

Code

```
1 awk 'BEGIN{ OFS="," } { print $1, $2 }' data.txt
```

Output

```
alice,10  
bob,20
```

Explanation

You can control output formatting by setting variables like **OFS** (Output Field Separator). This is typically done in a **BEGIN** block.



Regex Patterns

File Content (names.txt)

```
alice  
bob  
carol  
brian
```

Code

```
1 awk '/^b/ { print }' names.txt
```

Output

```
bob  
brian
```

Use regular expressions enclosed in slashes (/regex/) as patterns. The action is performed only on matching lines. The pattern /^b/ matches any line starting with 'b'.



print vs printf

File Content (scores.txt)

```
alice 10  
bob 200  
carol 5
```

Code

```
1 awk '{ printf("%-6s %5d\n", $1, $2) }' scores.txt
```

Output

```
alice    10  
bob     200  
carol     5
```

Explanation

printf provides C-style formatted printing for creating aligned output. `%-6s` is a left-aligned string padded to 6 characters.



BEGIN / END Blocks

File Content (data.txt)

```
alice 10  
bob 20
```

Code

```
1 awk 'BEGIN{ print "name score" } { print } END{ print "--  
done --" }' data.txt
```

Output

```
name score  
alice 10  
bob 20  
-- done --
```

Explanation

BEGIN runs once before processing files (for headers). **END** runs once after all lines are processed (for summaries).



Variables and Arithmetic

Code

```
1 awk '{ sum += $1 } END { print "sum=", sum, "avg=", sum/NR }' numbers.txt
```

File Content (numbers.txt)

```
10  
20  
5
```

Output

```
sum= 35 avg= 11.6667
```

Explanation

`awk` supports variables and arithmetic. This example calculates the sum and average of numbers, printing the final result in the **END** block.



String Functions

File Content (name.txt)

Carol

Code

```
1 awk '{ s = tolower($1); print s, length(s), substr(s,2,3) }  
    ' name.txt
```

Output

carol 5 aro

Explanation

awk has many built-in string functions, including `tolower()`, `length()`, and `substr(string, start, length)`.



Conditionals (if/else)

Code

```
1 awk '{ if ($2 >= 15) grade="A"; else grade="B"; print $1, grade }' scores.txt
```

File Content (scores.txt)

```
alice 10  
bob 20  
carol 15
```

Output

```
alice B  
bob A  
carol A
```

Explanation

Standard **if-else** statements can be used inside an action block to apply logic to each line based on its content.



Loops: for, while

File Content (numbers.txt)

```
1 2 3 4 5
```

Code

```
1 awk '{ s=0; for(i=1;i<=NF;i++) s+= $i; print "sum=",s }' numbers.txt
```

Output

```
sum= 15
```

Explanation

A **for** loop can iterate through all fields in a line (from **i=1** to **i<=NF**) to perform row-wise calculations.



Associative Arrays

Code

```
1 awk '{ count[$1]++ } END { for (k in count) print k, count[k] }' letters.txt
```

File Content (letters.txt)

```
a  
b  
a  
c  
b  
a
```

Output

```
a 3  
b 2  
c 1
```

Explanation

`awk`'s arrays are associative (like hash maps), using strings as keys. This makes them ideal for counting occurrences.



Group By and Sum

Code

```
1 awk '{ sum[$1] += $3 } END { for (dept in sum) print dept, sum[dept] }' records.txt
```

File Content (records.txt)

```
sales alice 10  
sales bob 20  
eng carol 7  
eng dave 3
```

Output

```
eng 10  
sales 30
```

Explanation

Associative arrays are perfect for "group by" operations. This example groups by department (\$1) and sums the associated values (\$3).



User-Defined Functions

Code

```
1 awk 'function avg(s,c){ return c? s/c : 0 } { s+=$1; c++ } END{ print "avg=", avg(s,c) }'  
    numbers.txt
```

File Content (numbers.txt)

```
10  
20  
30
```

Output

```
avg= 20
```

Explanation

You can define your own functions to encapsulate logic. Here, an `avg()` function is defined and then called from the **END** block.



Passing Shell Variables with -v

Code

```
1 threshold=15
2 awk -v t=$threshold '{ if ($2 >= t) print $1, ">=", t }' scores.txt
```

File Content (scores.txt)

```
alice 10
bob 20
carol 15
```

Output

```
bob >= 15
carol >= 15
```

Explanation

The **-v** option safely passes a shell variable into an **awk** script. It assigns the shell variable's value to an **awk** variable (here, **t**).



Column Totals by Index

Code

```
1 awk '{ for(i=1;i<=NF;i++) col[i]+=$i } END{ for(i=1;i<=NF;i++) print "col" i "=", col[i] }' data.txt
```

File Content (data.txt)

```
1 2 3
4 5 6
7 8 9
```

Output

```
col1= 12
col2= 15
col3= 18
```

Explanation

This command calculates the sum of each column. For each line, it loops through all fields and adds the value of each field (\$i) to an array element (col[i]) corresponding to its column number. The **END** block then prints the totals.



Group Averages

Code

```
1 awk '{ sum[$1]+=$2; cnt[$1]++ } END { for (g in sum) print g, sum[g]/cnt[g] }' data.txt
```

File Content (data.txt)

```
A 10  
A 20  
B 5  
B 15
```

Output

```
A 15  
B 10
```

Explanation

This is a common data aggregation pattern. It uses two associative arrays: one to hold the sum for each group (`sum[$1]`) and another to hold the count (`cnt[$1]`). In the **END** block, it iterates through the groups and divides the sum by the count to find the average for each.



Practice: filter + format

Code

```
1 cat << 'DATA' | awk '$3=="OK" { printf("%-5s %-5s %s\n", $1, $2, $3) }'  
2 ID1 12 OK  
3 ID2 7 FAIL  
4 ID3 33 OK  
5 DATA
```

Output

```
ID1 12 OK  
ID3 33 OK
```



The -f Flag: Specifying a Script File

Purpose

For complex logic, writing the entire script on the command line is impractical. The -f flag tells AWK to read its program from a specified file.



The -f Flag: Specifying a Script File

1. The Script File (report.awk):

```
1 # report.awk: A script to calculate average salary.
2
3 # This block runs once at the very beginning.
4 BEGIN {
5     FS = "," # Set Field Separator for CSV
6     print "--- Employee Salary Report ---"
7 }
8
9 # This action runs for every line after the first (skips header).
10 NR > 1 {
11     # NR is the current line number.
12     total_salary += $4 # Add salary (4th field) to total
13     employee_count++
14 }
15
16 # This block runs once after all lines have been processed.
17 END {
18     print "-----"
19     printf "Average Salary: $%.2f\n", total_salary / employee_count
20 }
21
```



The -f Flag: Specifying a Script File

2. The Command to Run the Script:

```
1 # Assume 'data.csv' has employee data.  
2 awk -f report.awk data.csv  
3
```



awk

Thank You for Listening!